

Project Title

Design and Implementation of a Mini Banking System using Layered Architecture

Problem Statement

In real-world banking systems, managing customer accounts and financial transactions requires proper structure, validation, and separation of responsibilities. Basic programs that handle deposits and withdrawals without organized design become difficult to maintain and extend. Therefore, there is a need to develop a structured Mini Banking System that simulates real-world banking operations such as account creation, deposits, withdrawals, and fund transfers. The system should ensure proper balance validation, exception handling, and maintain transaction history. This project aims to design and implement a well-structured banking application using object-oriented programming principles and layered architecture to improve maintainability, scalability, and clarity of code.

Target Users

- Bank administrators
- Customers

Core Features

- Create and manage customer accounts
- Support multiple account types (Savings and Current)
- Deposit and withdraw money
- Fund transfer between accounts
- Maintain transaction history
- Implement custom exception handling for invalid operations
- Input validation for secure transactions

OOP Concepts to be used

- **Abstraction** – Abstract Account class
- **Inheritance** – Savings Account and Current Account extending Account
- **Polymorphism** – Common handling of different account types
- **Encapsulation** – Private attributes with controlled access
- **Exception Handling** – Custom and built-in exceptions
- **Collections** – Use of List to manage account

Proposed Architecture

The system will follow a layered architecture consisting of Model, Service, Repository, and Exception layers. The Model layer will define account-related classes. The Service layer will handle business logic such as validation and transaction processing. The Repository layer will manage data storage using collections and file handling. The Exception layer will define custom exceptions to handle error scenarios effectively. This separation ensures modularity and clean design.
